

UNIDAD 05

Funciones de agregación y agrupamiento

1. Introducción a la agregación de datos

En el ámbito del desarrollo de sistemas de información y el análisis de datos, las funciones de agregación representan un pilar fundamental para transformar conjuntos extensos de datos primarios en información resumida y significativa. Una función de agregación (Aggregate Function) opera intrínsecamente sobre un conjunto de filas, o un subconjunto de ellas definido por una operación de agrupamiento, y tiene como propósito fundamental devolver un único valor de resumen que describe ese conjunto.¹ Estas operaciones son esenciales para el análisis descriptivo, facilitando la transición desde la granularidad de las transacciones individuales hasta la granularidad de los conjuntos o grupos de interés.

La agregación de datos se conceptualiza, de manera general, como un proceso de tres fases lógicas: la división, la aplicación y la combinación (Split, Apply, Combine).³ Primero, el conjunto de datos es segmentado en fragmentos discretos basados en los valores de las variables seleccionadas para la agrupación (fase de Split). Segundo, se aplica la función de agregado (e.g., SUM, AVG) a cada uno de estos segmentos o grupos de forma independiente (fase de Apply). Finalmente, los resultados calculados para cada grupo son consolidados en un conjunto de resultados unificado (fase de Combine), donde cada tupla representa el resumen de un grupo.

2. Análisis detallado de las funciones de agregación

Transact-SQL (T-SQL) en SQL Server soporta un conjunto de funciones de agregación estándar que pueden aplicarse a expresiones de columna. Opcionalmente, estas funciones pueden incluir el modificador DISTINCT, lo que permite que la función opere únicamente sobre los valores únicos dentro del conjunto o grupo, un detalle semántico crucial para evitar el conteo o la suma de duplicidades accidentales.⁴

2.1. Funciones cuantitativas: SUM y AVG

SUM (Suma)

La función SUM calcula la suma algebraica total de los valores numéricos contenidos en una columna específica.¹ Es imperativo que la columna de entrada sea de un tipo de dato numérico.

Sintaxis: SUM (expresión)

La inclusión del modificador DISTINCT en la función SUM provoca una diferencia semántica significativa. Por ejemplo, si se calcula SUM(Price), se suman todos los valores de precio en el conjunto. Sin embargo, si se utiliza SUM(DISTINCT Price), la función primero elimina los precios duplicados y solo después realiza la suma sobre los valores únicos restantes. Esta diferenciación es vital en contextos de auditoría o análisis donde la unicidad del valor de la métrica es requerida.

AVG (Promedio)

La función AVG es utilizada para calcular el valor promedio o media aritmética del conjunto de valores numéricos de una columna.¹

Sintaxis: AVG (expresión)⁵

Al igual que en SUM, AVG también acepta el modificador DISTINCT. El cálculo de AVG(DISTINCT expression) produce un resultado inherentemente diferente al de AVG(expression). El primero calcula el promedio matemático de los valores únicos encontrados, mientras que el segundo calcula el promedio de la totalidad de los valores, incluyendo todas las instancias duplicadas.⁶ Esta distinción puede ser decisiva cuando se analizan distribuciones de datos donde ciertos valores ocurren con mucha mayor frecuencia que otros.

2.2. Funciones de extremos: MIN y MAX

Las funciones MIN y MAX son utilizadas para identificar el valor más bajo y el valor más alto, respectivamente, dentro del conjunto de datos de entrada.¹

Estas funciones demuestran una aplicabilidad amplia que excede las columnas numéricas. Pueden aplicarse efectivamente a:

1. **Fechas:** MIN devuelve la fecha más antigua; MAX devuelve la fecha más reciente.
2. **Cadenas de Caracteres:** MIN y MAX devuelven el valor que es alfabéticamente menor o mayor, respectivamente, basándose en la secuencia de intercalación (collation) definida en la base de datos de SQL Server.

2.3. La Función COUNT y la semántica de NULLs:

La función COUNT requiere un manejo preciso y detallado de su sintaxis, dado que su comportamiento respecto a los valores nulos (NULL) es fundamentalmente diferente al de otras funciones de agregado, impactando directamente en la integridad de los resultados.

COUNT(*): Recuento de Filas (Tuplas)

Esta forma de la función cuenta el número total de filas o tuplas presentes en el grupo o el conjunto de resultados.⁷ El aspecto más relevante de COUNT(*) es su manejo de los valores NULL: COUNT(*) **no ignora** los valores NULL en ninguna columna, puesto que su propósito es medir la existencia de la tupla como entidad, independientemente del contenido de sus atributos.⁸ La sintaxis COUNT(1) es semánticamente idéntica a COUNT(*) ya que ambos cuentan una expresión no nula por cada fila.¹⁰

COUNT(expresión) / COUNT(Columna): Recuento de Valores Existentes

Esta sintaxis cuenta el número de filas donde la expresión (generalmente el nombre de una columna) evalúa a un valor **no nulo**.² A diferencia de COUNT(*), la función COUNT(columna) **ignora** explícitamente cualquier valor NULL presente en esa columna durante el cálculo.⁹

COUNT(DISTINCT expresión): Recuento de Valores Únicos No Nulos

Esta forma cuenta exclusivamente la cantidad de valores únicos (distintos) y, además, no nulos dentro del conjunto o grupo analizado.⁴

Diagnóstico de Integridad de Datos mediante COUNT

La diferencia entre los resultados obtenidos por COUNT(*) y COUNT(Columna) ofrece una herramienta de diagnóstico cuantitativo inmediato para evaluar la calidad y la integridad de los datos en un atributo específico.

La lógica de esta metodología es la siguiente: si COUNT(*) mide el total de tuplas en un conjunto⁸, y COUNT(Columna) mide solo aquellas tuplas que poseen un valor definido para esa columna¹¹, entonces, la diferencia numérica entre estos dos resultados es precisamente el número de filas que contienen un valor NULL para dicho atributo. Este contraste es esencial para los analistas de sistemas, ya que permite identificar de forma programática la proporción de campos que, aunque pueden ser opcionales en el diseño de la base de datos, están quedando vacíos en los sistemas transaccionales, cuantificando así el nivel de incompletitud de los datos.

La siguiente tabla sintetiza la semántica de la agregación respecto al manejo de valores nulos:

Función de Agregación	Comportamiento con NULLs	Implicación Semántica	Resultado en Conjunto Vacío
SUM(Columna)	Ignora NULLs	Opera solo sobre valores definidos.	NULL
AVG(Columna)	Ignora NULLs	Excluye NULLs del numerador y del denominador.	NULL
MIN(Columna)/MAX(Columna)	Ignora NULLs	NULL no es considerado un extremo.	NULL
COUNT(Columna)	Ignora NULLs	Mide la integridad de los datos (valores existentes).	0
COUNT(*)/COUNT(1)	No ignora NULLs	Mide el tamaño físico del conjunto de filas (tuplas).	0

Tabla: Semántica del Manejo de Valores NULL en Funciones de Agregación Estándar

3. La cláusula GROUP BY: segmentación y agrupación

La cláusula GROUP BY es un componente fundamental de T-SQL que permite al analista elevar el nivel de abstracción y análisis de los datos. Su función principal es dividir el resultado de una consulta en grupos de filas, típicamente para aplicar una o más funciones de agregación a cada grupo.¹²

3.1. Propósito y funcionamiento

El propósito de GROUP BY es segmentar un conjunto de datos en función de la igualdad de valores en una o más columnas especificadas. Cada grupo resultante es tratado como una unidad para la cual se calculará un valor de resumen a través de las funciones de agregación.³

La aplicación de esta cláusula tiene un impacto directo en la granularidad. Transforma el conjunto de filas detalladas en un conjunto de filas resumidas, donde cada fila de salida representa un grupo único definido por la combinación de los valores en las columnas de GROUP BY.¹³

3.2. Reglas de sintaxis y restricciones de integridad

Una regla sintáctica estricta y crucial en SQL Server rige el uso de GROUP BY: cualquier columna que se liste en la cláusula SELECT que no esté directamente involucrada en una función de agregación **debe** estar incluida en la cláusula GROUP BY.¹⁴

Esta restricción es impuesta por el motor de base de datos para garantizar la coherencia semántica. Si un conjunto de filas se agrupa por *Departamento* (resultando en una fila por departamento), y se intenta seleccionar también el *NombreEmpleado*, el motor no tiene manera de decidir qué nombre individual de los múltiples empleados dentro del grupo de ese departamento debe mostrar en la fila de resumen. Al obligar al uso de GROUP BY, se asegura que todas las columnas no agregadas en el SELECT sean, por definición, las claves que definen la granularidad del resumen.

Cuando se agrupa por múltiples columnas (GROUP BY Columna1, Columna2), se establece una **agrupación compuesta o jerárquica**. Un grupo solo se forma si los valores de *todas* las columnas especificadas coinciden. Por ejemplo, agrupar por (*Región, Ciudad*) generará un grupo único para cada combinación específica de región y ciudad.¹³

3.3. Granularidad y la compensación por detalle

La aplicación exitosa de la cláusula GROUP BY en una consulta representa intrínsecamente una pérdida controlada de detalle informativo. La granularidad de la información se eleva (por ejemplo, de la línea de pedido individual a la venta total por cliente), lo cual es deseable para reportes y análisis de alto nivel. Sin embargo, al colapsar filas de entrada en una única fila de resumen, la información específica de cada una de las filas originales que no fue objeto de agregación se vuelve irrecuperable en el conjunto de resultados agrupados.¹⁴

Este fenómeno establece una compensación fundamental en el diseño de consultas: se gana eficiencia y resumen, pero se sacrifica el detalle a nivel de registro. Por lo tanto, el diseño de la consulta debe comenzar siempre por determinar el nivel de granularidad deseado para el reporte final, asegurándose de que todas las columnas no agregadas reflejen correctamente ese nivel (es decir, que sean las claves de agrupación).

4. Filtrado de grupos agregados: la cláusula HAVING

La cláusula HAVING se introdujo en el estándar SQL precisamente para subsanar una limitación operativa de la cláusula WHERE: la incapacidad de esta última para aplicar filtros basados en los resultados de funciones de agregado.¹⁶

4.1. Definición y Necesidad Operacional

HAVING especifica una condición de búsqueda que es aplicada a un grupo o a un valor agregado.¹⁶ Su

función es actuar como un filtro que opera sobre los resultados ya producidos por las fases de agrupación y agregación.¹⁷

Es el mecanismo exclusivo en T-SQL que permite al desarrollador o analista filtrar el conjunto de resultados basado en una métrica resumida. Por ejemplo, si se desea analizar solo aquellos departamentos donde la SUM(Sueldos) total supera un umbral determinado, esta condición debe ser manejada obligatoriamente por HAVING, ya que el valor de la suma no existe hasta después de que se completa la operación GROUP BY.¹⁶

4.2. Sintaxis y posición en la consulta (T-SQL)

La cláusula HAVING tiene una posición fija y estricta en el orden sintáctico de la sentencia SELECT de T-SQL. Debe colocarse obligatoriamente después de la cláusula GROUP BY y antes de la cláusula ORDER BY.¹⁶

La sintaxis general para una consulta con agrupamiento y filtrado de grupos es la siguiente:

```
SELECT
    Columna_Agrupacion,
    FUNCION_AGREGADO(Columna_Valor)
FROM
    Tabla
GROUP BY
    Columna_Agrupacion
HAVING
    Condicion_sobre_Agregado;
```

Es importante señalar que las condiciones dentro de HAVING pueden referenciar tanto a las columnas utilizadas en la agrupación como al resultado de las funciones de agregado. Además, SQL Server impone restricciones de tipo de dato: los tipos text, image y ntext no pueden ser utilizados en la cláusula HAVING.¹⁶

4.3. Ejemplo Práctico

Utilizando el esquema de ejemplo AdventureWorks, se puede ilustrar la aplicación del filtrado post-agregación.

Escenario: Determinar qué pedidos de venta (SalesOrderID) tienen un subtotal (suma de las líneas de pedido, LineTotal) que excede el valor de \$100,000.00.

```
-- Ejemplo de Filtrado Post-Agregación
USE AdventureWorks2022;
GO
SELECT
    SalesOrderID,
    SUM(LineTotal) AS SubTotal
FROM
    Sales.SalesOrderDetail
GROUP BY
    SalesOrderID
HAVING
    SUM(LineTotal) > 100000.00 -- Condición aplicada al valor SUM() del grupo
ORDER BY
    SalesOrderID;
```

El motor de la consulta primero agrupa todas las líneas por SalesOrderID y calcula el SubTotal para cada pedido. Posteriormente, la cláusula HAVING se ejecuta, descartando aquellos grupos cuyo valor calculado de SUM(LineTotal) no alcanza el umbral de \$100,000.00.¹⁶

5. Diferencia fundamental y secuencia de procesamiento: WHERE vs. HAVING

La distinción entre WHERE y HAVING es uno de los conceptos más críticos y frecuentemente malentendidos en el desarrollo de consultas SQL, ya que define la fase lógica en la que ocurre el filtrado de datos. La diferencia clave es operativa y se basa en el momento de la ejecución dentro del ciclo de vida de la consulta.¹⁶

5.1. Distinción operacional basada en el objeto de filtrado

La distinción se resume en la naturaleza del objeto que se somete a la condición de filtrado:

- **Cláusula WHERE:** Esta cláusula se ejecuta **antes** de la agrupación (GROUP BY). Su objetivo es reducir el número de **filas individuales** que serán procesadas por la función de agrupación.¹⁶ Por su posición temprana en la ejecución, WHERE no puede referenciar ni utilizar funciones de agregación.
- **Cláusula HAVING:** Esta cláusula se ejecuta **después** de la agrupación y agregación (GROUP BY). Su objetivo es reducir el número de **grupos de filas** (es decir, las tuplas de resumen) que se incluirán en el resultado final.¹⁶ HAVING debe basar su condición en una función agregada o en una columna de agrupación.¹⁹

La siguiente tabla compara las características de estas dos cláusulas de filtrado:

Criterio	Cláusula WHERE	Cláusula HAVING
Objeto de Filtrado	Filas Individuales (Tuplas)	Grupos de Filas (Resultados Agregados)
Fase de Ejecución	Antes de GROUP BY	Después de GROUP BY
Uso de Funciones Agregadas	No Permitido	Permitido y, a menudo, Requerido
Impacto en el Set de Datos	Reduce la Entrada para la Agregación	Reduce la Salida de la Agregación

Tabla: Comparación Operacional: Cláusula WHERE vs. Cláusula HAVING

5.2. El orden lógico de ejecución de consultas T-SQL

Para cualquier sistema que dependa de la optimización del plan de consulta, es imperativo comprender la secuencia en que el motor de base de datos de SQL Server procesa lógicamente las cláusulas. Este orden determina la semántica y, crucialmente, el rendimiento de la consulta.²⁰

El orden lógico de procesamiento de una sentencia SELECT con agrupamiento es:

1. **FROM y JOINS:** Determina y combina el conjunto inicial de filas de las tablas fuente.
2. **WHERE:** Filtra las filas individuales resultantes de la unión o tabla fuente.
3. **GROUP BY:** Agrupa las filas ya filtradas.
4. **Funciones de Agregación:** Se calculan los valores de resumen para cada grupo.
5. **HAVING:** Filtra los grupos resultantes basándose en el cálculo de agregado.
6. **SELECT:** Proyecta las columnas finales y los agregados.
7. **ORDER BY:** Ordena el conjunto de resultados final.

5.3. Eficiencia causal y optimización de consultas

La secuencia de ejecución de consultas impone una regla rigurosa para la optimización del rendimiento. Dado que la cláusula WHERE se ejecuta antes que la operación GROUP BY, cualquier condición que pueda ser aplicada a nivel de fila individual debe residir prioritariamente en WHERE.¹⁸

Esta metodología se basa en el principio de que la operación de agrupamiento y el cálculo de funciones de agregado son, inherentemente, algunas de las operaciones más costosas en términos de consumo de

recursos y tiempo de procesamiento. Al aplicar los filtros de fila en la fase WHERE, se garantiza que el tamaño del conjunto de datos se minimice antes de que se inicie la costosa operación de GROUP BY.

Si un analista coloca un filtro a nivel de fila en la cláusula HAVING (por ejemplo, si intenta filtrar por una condición de fecha sin agregación), el motor de SQL se vería forzado a realizar primero la costosa agregación sobre el conjunto de datos completo, solo para que el HAVING descarte posteriormente los grupos que no cumplen la condición. El uso apropiado del filtro WHERE en la etapa temprana de la consulta es, por lo tanto, la palanca de optimización inicial más potente en cualquier sentencia que involucre la agregación de datos.

6. Caso de práctico

Para ejemplificar la aplicación práctica de WHERE, GROUP BY y HAVING en un contexto de T-SQL, se presenta un escenario con un caso hipotético.

6.1. Definición del escenario

Se dispone de una tabla Ventas que registra transacciones (ProductID, Region, Date, Amount). El requerimiento analítico es:

1. Considerar únicamente las ventas realizadas durante el **año 2023** (Filtro a nivel de fila).
2. Agrupar los resultados por Region y ProductID.
3. Identificar aquellos grupos de producto-región donde el **Total de Ventas (SUM(Amount)) superó los \$100,000.00** (Filtro a nivel de grupo).

6.2. Consulta

```
-- Simulación de tabla de Ventas
-- CREATE TABLE Ventas (ProductID INT, Region NVARCHAR(50), Date DATE, Amount
--   DECIMAL(18, 2));

SELECT Region, ProductID, SUM(Amount) AS TotalVentas_2023
FROM Ventas
WHERE YEAR(Date) = 2023 -- FASE 1: Filtrado de filas (pre-agregación)
GROUP BY Region, ProductID -- FASE 2: Agrupación de las filas filtradas
HAVING SUM(Amount) > 100000.00 -- FASE 3: Filtrado de grupos (post-agregación)
ORDER BY Region, TotalVentas_2023 DESC;
```

6.3. Análisis de la ejecución secuencial del ejemplo

El motor de SQL Server procesa la consulta de la siguiente manera, asegurando la eficiencia y el cumplimiento semántico:

1. **Filtrado Inicial (WHERE):** El proceso comienza con la selección de la tabla Ventas. Inmediatamente, se aplica la condición `WHERE YEAR(Date) = 2023`. En esta fase, todas las filas correspondientes a

ventas de años distintos a 2023 son descartadas antes de cualquier cálculo de resumen. Esto minimiza el volumen de datos que ingresa al siguiente paso.¹⁸

2. **Agrupamiento (GROUP BY) y Agregación:** Las filas restantes (el subconjunto optimizado de 2023) son segmentadas en grupos definidos por la combinación única de *Region* y *ProductID*. Para cada grupo, se calcula el SUM(Amount), generando el valor *TotalVentas_2023*.
3. **Filtrado de Grupos (HAVING):** Una vez calculados los totales por grupo, la cláusula HAVING entra en acción. Evalúa la condición `SUM(Amount) > 100000.00`. Aquellos grupos (es decir, aquellas combinaciones específicas de *Región* y *Producto*) cuyo total agregado no supera los \$100,000.00 son descartados del conjunto de resultados final. Este filtrado garantiza que solo los grupos de alto rendimiento sean presentados.¹⁷
4. **Proyección (SELECT) y Ordenación (ORDER BY):** Finalmente, las columnas *Region*, *ProductID*, y el *TotalVentas_2023* se proyectan y se ordenan.

7. Recomendaciones metodológicas

Para asegurar la precisión y la eficiencia en el desarrollo de consultas con agregación, se establecen las siguientes recomendaciones metodológicas:

- **Priorizar el Filtrado Temprano (WHERE):** Siempre se debe utilizar la cláusula WHERE para aplicar cualquier condición que pueda ser evaluada a nivel de fila. Esta práctica garantiza que la operación inherentemente costosa de GROUP BY y agregación se ejecute sobre el subconjunto de datos más pequeño posible, optimizando el rendimiento general de la consulta.
- **Aplicación Exclusiva de HAVING:** La cláusula HAVING debe reservarse estrictamente para condiciones de filtrado que dependen del resultado de una función de agregado ya calculada.
- **Uso de COUNT Consistente:** Para auditorías de integridad de datos, se recomienda siempre ejecutar COUNT(*) en paralelo con COUNT(Columna) para diagnosticar la densidad de valores nulos. Si el objetivo es simplemente medir el tamaño del registro lógico, COUNT(*) o COUNT(1) son las sintaxis más claras y eficientes.

Referencias

1. COUNT, AVG, SUM, MIN, MAX Functions in SQL - Codedamn, fecha de acceso: octubre 7, 2025, <https://codedamn.com/news/sql/count-avg-sum-min-max-functions-in-sql>
2. Funciones de agregación (Creador de SQL) - IBM, fecha de acceso: octubre 7, 2025, <https://www.ibm.com/docs/es/iis/11.5.0?topic=grid-aggregation-functions>
3. How to Use GROUP BY and HAVING in SQL - DataCamp, fecha de acceso: octubre 7, 2025, <https://www.datacamp.com/tutorial/group-by-having-clause-sql>
4. COUNT (Transact-SQL) - SQL Server - Microsoft Learn, fecha de acceso: octubre 7, 2025, <https://learn.microsoft.com/en-us/sql/t-sql/functions/count-transact-sql?view=sql-server-ver17>
5. AVG (Transact-SQL) - SQL Server | Microsoft Learn, fecha de acceso: octubre 7, 2025, <https://learn.microsoft.com/es-es/sql/t-sql/functions/avg-transact-sql?view=sql-server-ver17>
6. La función SQL AVG() explicada con ejemplos | LearnSQL.es, fecha de acceso: octubre 7, 2025, <https://learnsql.es/blog/la-funcion-sql-avg-explicada-con-ejemplos/>
7. Count(*) vs Count(ID)? ¿Cuál es la diferencia? : r/SQL - Reddit, fecha de acceso: octubre 7, 2025, https://www.reddit.com/r/SQL/comments/1bp0mtw/count_vs_countid_what_is_the_difference/?tl=es-es
8. COUNT (SQL) | InterSystems SQL Reference | InterSystems IRIS Data Platform 2025.2, fecha de acceso: octubre 7, 2025, https://docs.intersystems.com/irislatest/csp/docbook/DocBook.UI.Page.cls?KEY=RSQL_count
9. NULL semantics - Azure Databricks - Microsoft Learn, fecha de acceso: octubre 7, 2025, <https://learn.microsoft.com/en-us/azure/databricks/sql/language-manual/sql-ref-null-semantics>
10. Count(*) vs Count(1) - SQL Server - Stack Overflow, fecha de acceso: octubre 7, 2025, <https://stackoverflow.com/questions/1221559/count-vs-count1-sql-server>
11. sql - count(*) vs count(column-name) - which is more correct? - Stack Overflow, fecha de acceso: octubre 7, 2025, <https://stackoverflow.com/questions/3003457/count-vs-countcolumn-name-which-is-more-correct>
12. GROUP BY (Transact-SQL) - SQL Server | Microsoft Learn, fecha de acceso: octubre 7, 2025, <https://learn.microsoft.com/en-us/sql/t-sql/queries/select-group-by-transact-sql?view=sql-server-ver17>
13. SQL GROUP BY - Everything You Need To Know - Sisense, fecha de acceso: octubre 7, 2025, <https://www.sisense.com/blog/everything-about-group-by/>
14. Funciones GROUP BY y Agregadas: Una visión completa | LearnSQL.es, fecha de acceso: octubre 7, 2025, <https://learnsql.es/blog/funciones-group-by-y-agregadas-una-vision-completa/>
15. SQL GROUP BY en varias columnas: Consejos y mejores prácticas - DataCamp, fecha de acceso: octubre 7, 2025, <https://www.datacamp.com/es/tutorial/sql-group-by-multiple-columns>
16. SQL HAVING Clause: The Ultimate Guide - DbVisualizer, fecha de acceso: octubre 7, 2025, <https://www.dbvis.com/thetable/sql-having-clause-the-ultimate-guide/>
17. SQL HAVING - Syntax, Use Cases, and Examples - Hightouch, fecha de acceso: octubre 7, 2025, <https://hightouch.com/sql-dictionary/sql-having>
18. Use HAVING and WHERE Clauses in the Same Query | Microsoft Learn, fecha de acceso: octubre 7, 2025, <https://learn.microsoft.com/en-us/ssms/visual-db-tools/use-having-and-where-clauses-in-the-same-query-visual-database-tools>
19. Sql Server para novatos | Funciones de agregación, Group by, Having | #6 - YouTube, fecha de acceso: octubre 7, 2025, https://www.youtube.com/watch?v=jW_dXPWnps
20. Execution sequence of Group By, Having and Where clause in SQL Server?, fecha de acceso: octubre 7, 2025, <https://stackoverflow.com/questions/1130062/execution-sequence-of-group-by-having-and-where-clause-in-sql-server>